

Hostel & Mess Complaint and Feedback System

Project Report

Student Name	_____ (fill in)
Registration Number	_____ (fill in)
Course	Programming in Java
Evaluation Type	Flipped Course Evaluation - Build Your Own Project
Project Domain	Console-based Java application (OOP + File I/O)
Submission Deadline	September 18, 2026

1. Introduction

Hostel and mess complaints in most colleges are still handled informally - a verbal word to the warden, a message in a group chat, or an entry in a physical register. This makes it difficult to know how long an issue has actually been open, whether anyone is responsible for it, or whether it was ever resolved. Recurring issues (electrical faults, mess hygiene, plumbing) can sit unresolved for days without anyone being alerted, and there is no structured channel for students to give feedback once an issue is addressed.

This project implements a **console-based Java application** that lets students log complaints against defined categories, gives hostel administration a dashboard to track and resolve them, and automatically **escalates** any complaint that remains unresolved past a configurable number of days. Once resolved, the student who filed the complaint can leave a rating and comment, closing the feedback loop. The project applies core Programming-in-Java concepts: object-oriented design (abstraction, inheritance, encapsulation), collections, exception handling, and file I/O - without depending on any external framework or database.

2. Problem Statement

Design and implement a system that allows hostel residents to log complaints related to mess and hostel facilities, allows administration to track and resolve these complaints in a structured way, automatically flags complaints that have gone unresolved for too long, and collects feedback from students once their complaint is resolved - all from a single command-line Java application with no external dependencies.

Target Users

- **Students** living in the hostel, who need to report issues and track their resolution.
- **Hostel administration / warden**, who needs a consolidated view of open complaints and visibility into which ones are overdue.

3. Functional Requirements

The system implements the following major functional modules:

- 1 **User Management (Authentication)** - student self-registration, student & admin login, with a default admin account seeded automatically on first run.
- 2 **Complaint Logging (Data Input & Processing)** - students file complaints with a category, priority, and description; input is validated (non-empty, length-limited) before being timestamped and persisted.
- 3 **Admin Complaint Management (CRUD)** - admins can view all complaints, filter by status, and update a complaint's status (PENDING → IN_PROGRESS → RESOLVED) with remarks.
- 4 **Automatic Escalation** - any complaint still open past a configurable threshold (default 3 days) is automatically flagged ESCALATED the moment the admin dashboard loads, or on demand, and the event is written to an audit log.
- 5 **Feedback Collection** - once a complaint is RESOLVED, the student who filed it can submit a 1-5 rating and an optional comment.
- 6 **Reporting / Analytics** - admins can view complaint counts by status and category, average resolution time in hours, and the current count of escalated complaints.

4. Non-functional Requirements

Requirement	How it is addressed
Usability	Simple numbered console menus for every role; input prompts are explicit about the expected format (e.g. r
Reliability	All persistence operations go through a single FileStorageUtil so a partially-written file cannot corrupt more t
Error Handling	A custom checked exception (InvalidInputException) is used throughout the service layer for validation failur
Maintainability	Layered package structure (model / service / ui / util) keeps business logic independent of the console UI, so
Performance	All data structures are in-memory ArrayLists loaded once at startup; for the expected scale of a single hoste
Logging / Monitoring	Every auto-escalation event is appended to data/escalation.log with a timestamp, giving administration an a
Security (noted limitation)	Only registered students can file complaints under their own ID and only view their own complaint history; h

5. System Architecture

The application follows a simple layered architecture so that the console UI, business rules, and persistence stay independent of each other:

- **Presentation layer** (com.hms.ui) - AuthMenu, StudentMenu, AdminMenu render console menus and collect input; they contain no business logic.
- **Service layer** (com.hms.service) - AuthService, ComplaintService, FeedbackService, ReportService, and NotificationLogger implement all business rules, including the escalation rule.
- **Domain model** (com.hms.model) - User (abstract) with Student and Admin subclasses, plus Complaint and Feedback and their supporting enums.
- **Utility layer** (com.hms.util) - FileStorageUtil (generic file read/write), IdGenerator (sequential human-readable IDs), and InvalidInputException.
- **Persistence** - plain pipe-delimited text files under data/, so the whole project runs with just a JDK and no database server.

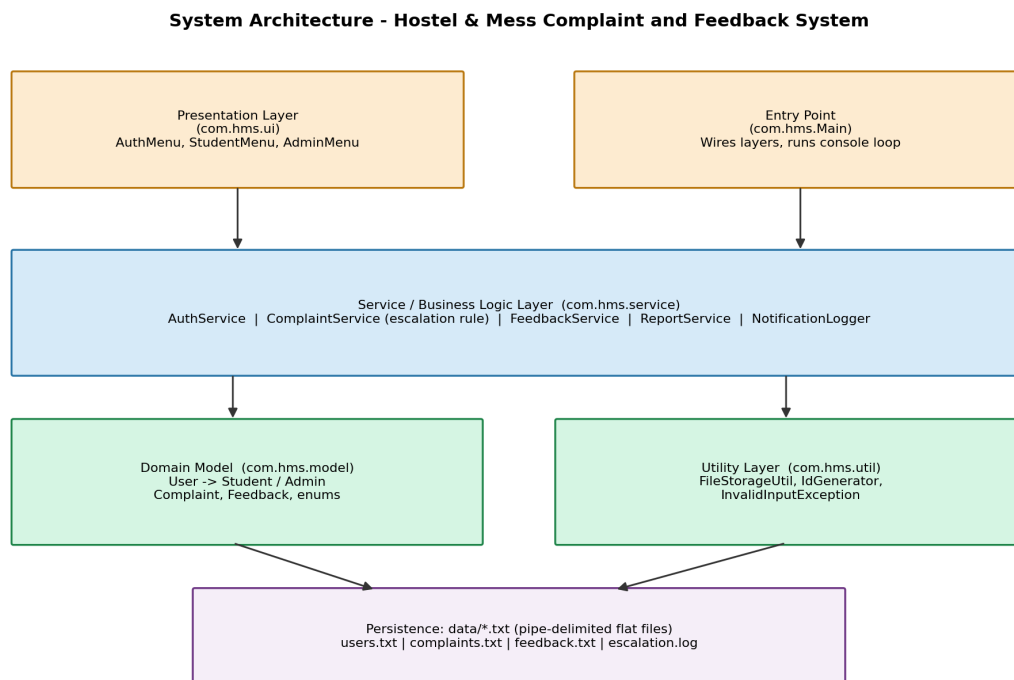


Figure 5.1 - Layered system architecture.

6. Design Diagrams

6.1 Use Case Diagram

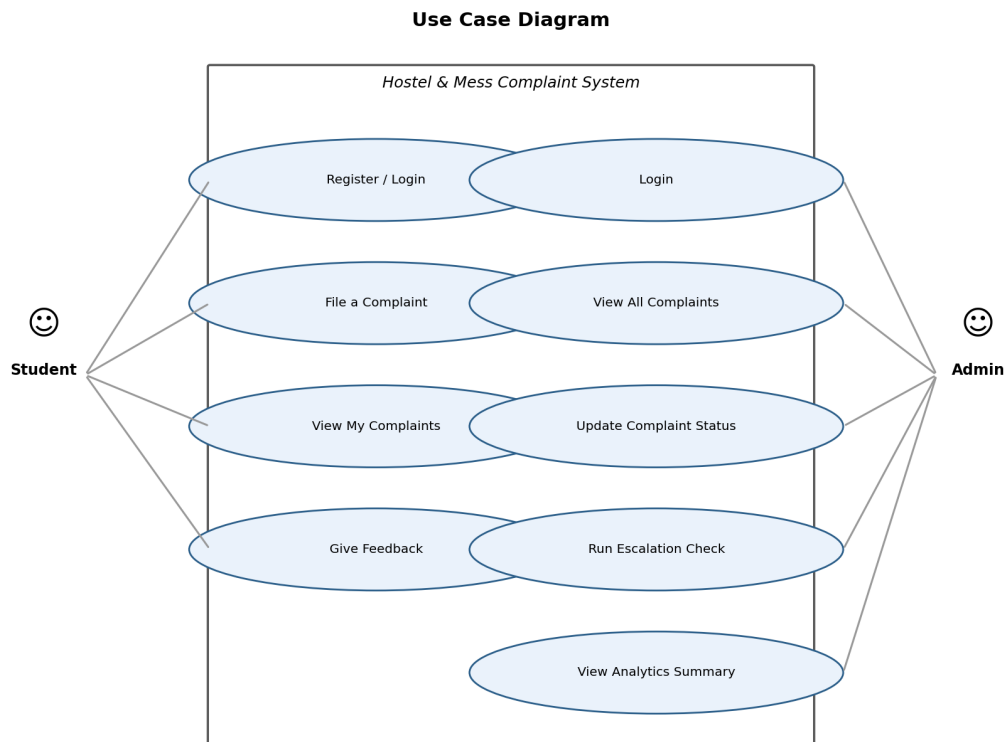


Figure 6.1 - Use cases for the Student and Admin actors.

6.2 Process / Workflow Diagram

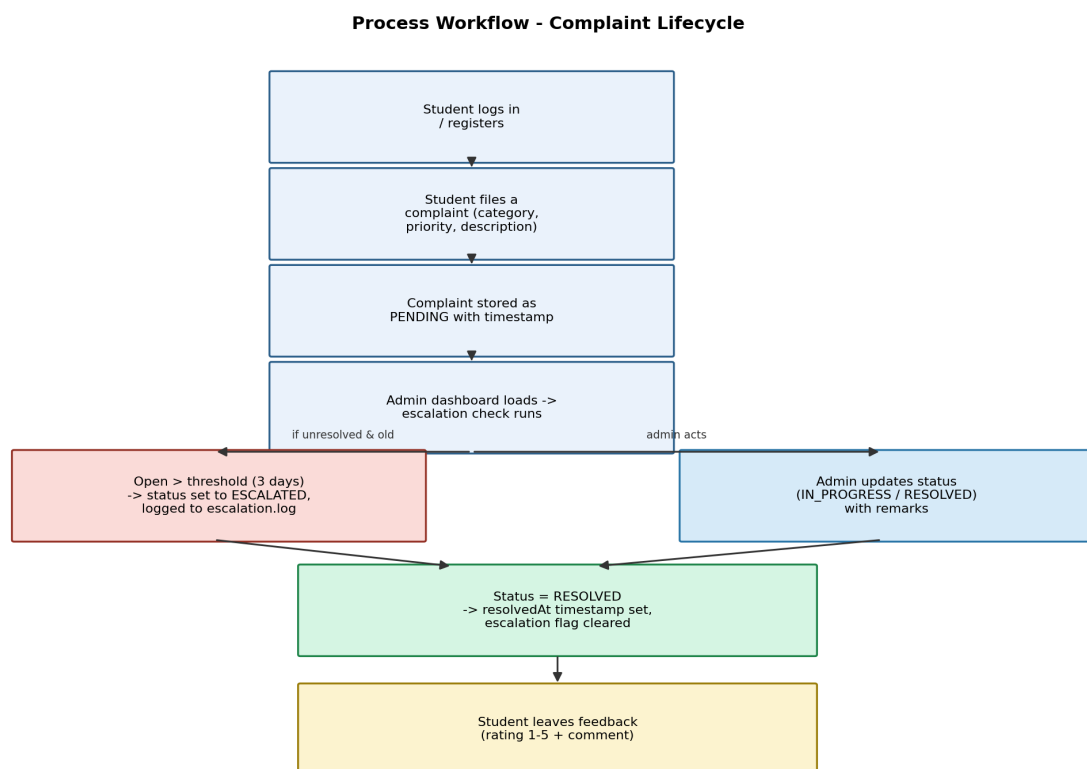


Figure 6.2 - Complaint lifecycle from filing through escalation, resolution, and feedback.

6.3 Class Diagram

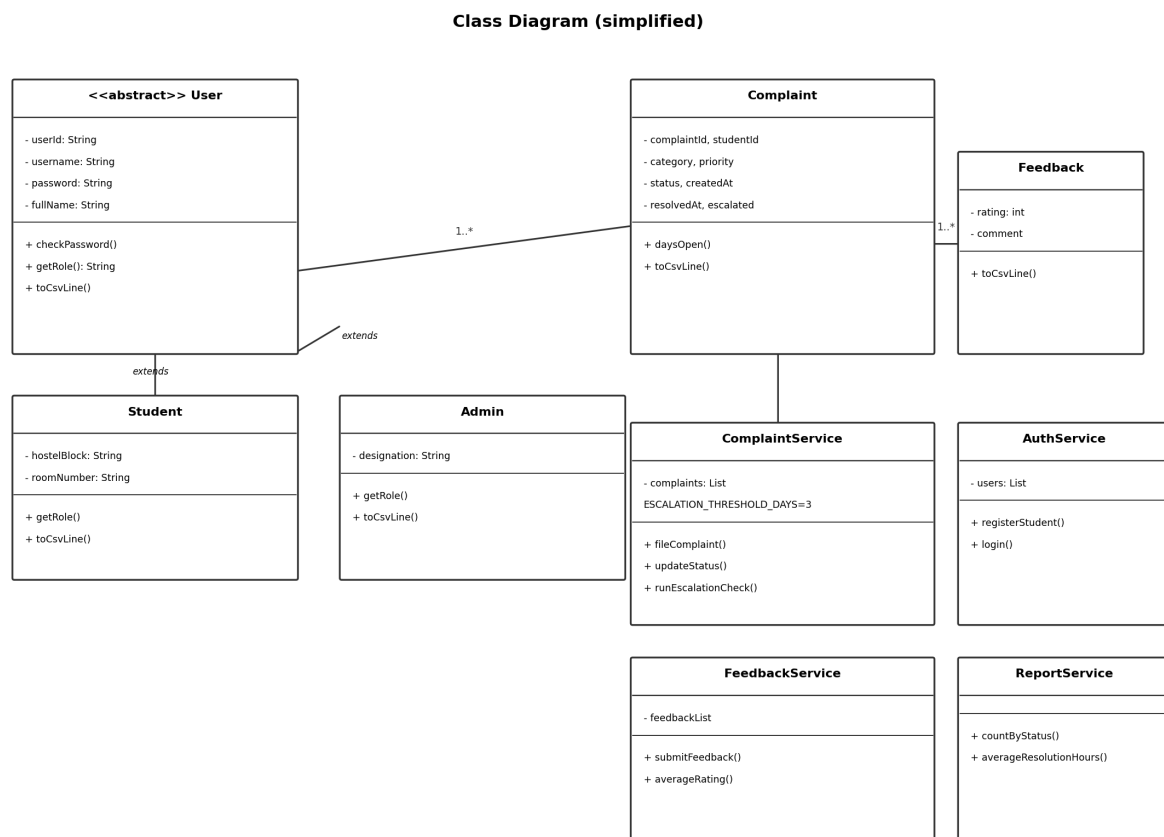


Figure 6.3 - Simplified class diagram showing the model and service layers.

6.4 Sequence Diagram

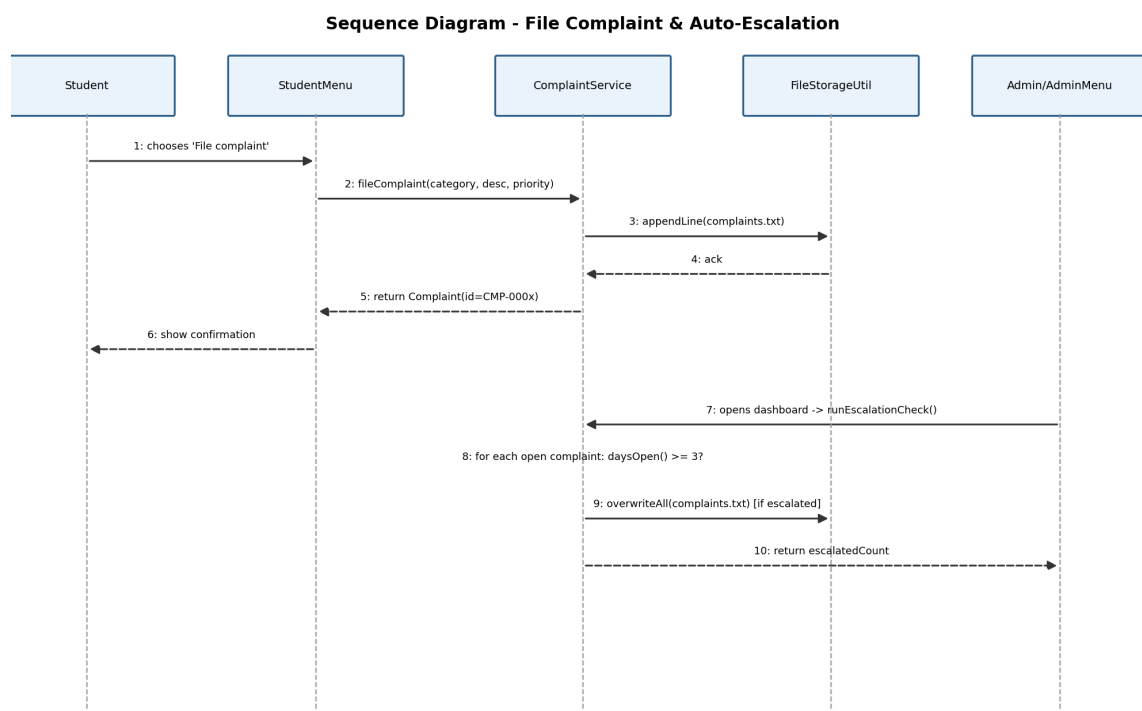
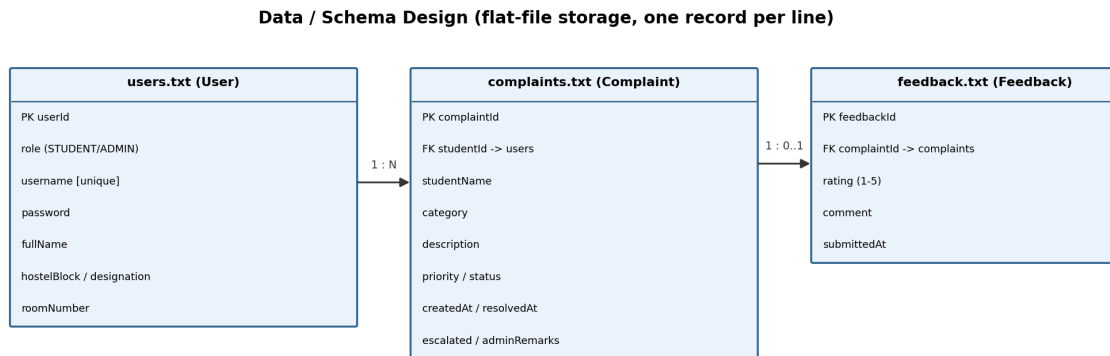


Figure 6.4 - Filing a complaint and the subsequent auto-escalation check.

6.5 ER / Schema Design

The project intentionally uses flat text files instead of a relational database, so that it can be compiled and run from the command line with no setup. The diagram below shows the same data modelled as it would map onto relational tables, including primary/foreign key relationships, so the design could be migrated to JDBC + a real database with minimal changes to the service layer.



escalation.log is an append-only audit log keyed by complaintid (not a relational table).

Figure 6.5 - Logical schema of the three data files and their relationships.

7. Design Decisions & Rationale

7.1 Why file-based storage instead of JDBC + a database

The submission requirements state the project must be "fully executable via the command line" with no GUI-based setup. Requiring a MySQL/PostgreSQL server and JDBC driver would add setup steps for the evaluator (installing and configuring a database) that have nothing to do with demonstrating Java concepts. Flat pipe-delimited files, accessed through a single `FileStorageUtil` class, give the same load/persist semantics a JDBC DAO layer would, while keeping the project runnable with just `javac` and `java`.

7.2 Why an abstract User class

Student and Admin share fields (ID, username, password, full name) and behaviour (password checking, CSV serialization) but differ in role-specific data (room/block vs. designation) and in what each role is allowed to do. Modelling this as an abstract base class with two concrete subclasses is a direct, teachable example of abstraction and inheritance - one of the "Correct application of subject concepts" expectations for the course.

7.3 Why the escalation threshold lives in one constant

`ComplaintService.ESCALATION_THRESHOLD_DAYS` is the single source of truth for "X days" in the escalation rule, checked with `java.time.Duration` between a complaint's `createdAt` and either now or its `resolvedAt`. Centralizing the threshold means a grader (or a real hostel) can retune escalation sensitivity without touching any other class.

7.4 Why a custom checked exception (`InvalidInputException`)

Using a single, explicit checked exception for all validation failures in the service layer keeps error handling consistent and visible at every call site (the compiler forces callers to handle it), rather than relying on unchecked exceptions or magic return values (e.g. `null` or `-1`) that are easy to silently ignore.

8. Implementation Details

The project is organised into 13 classes across four packages, all under the root package **com.hms**, using only the Java standard library (`java.time`, `java.nio.file`, `java.util`). No external JAR dependencies are required.

Package	Classes	Responsibility
com.hms	Main	Wires up all services and menus; runs the top-level login loop.
com.hms.model	User, Student, Admin, Complaint, Feedback, Category, Brand, ComplaintStatus	Domain entities and CSV serialization.
com.hms.service	AuthService, ComplaintService, FeedbackService, ReportsService, RegistrationService	Business logic: authentication, complaint CRUD + escalation, feedback, and reports generation.
com.hms.ui	AuthMenu, StudentMenu, AdminMenu	Console menus per role; input collection and display only.
com.hms.util	FileStorageUtil, IdGenerator, InvalidInputException	Support utilities: file I/O, sequential ID generation, and validation-error signalling.

Key algorithm - auto-escalation check (`ComplaintService.runEscalationCheck`)

```
public int runEscalationCheck() {
    LocalDateTime now = LocalDateTime.now();
    int escalatedCount = 0;
    for (Complaint c : complaints) {
        boolean stillOpen = c.getStatus() == ComplaintStatus.PENDING
            || c.getStatus() == ComplaintStatus.IN_PROGRESS;
        if (stillOpen && !c.isEscalated()
            && c.daysOpen(now) >= ESCALATION_THRESHOLD_DAYS) {
            c.setEscalated(true);
            escalatedCount++;
        }
    }
    return escalatedCount;
}
```

```

        c.setStatus(ComplaintStatus.ESCALATED);
        escalatedCount++;
        logger.logEscalation(c.getComplaintId(), c.getCategory().name(), c.daysOpen(now));
    }
}
if (escalatedCount > 0) persistAll();
return escalatedCount;
}

```

9. Screenshots / Results

As this is a command-line application, "screenshots" are captured as console transcripts of real runs. The full source and a step-by-step manual test procedure are in the repository README.

9.1 Student session - registration, login, filing a complaint

[setup] No admin account found - created default admin (username: admin / password: admin123).
 Change this after first login in a real deployment.
 Starting Hostel & Mess Complaint and Feedback System...

```

===== Hostel & Mess Complaint System =====
1. Login
2. Register as Student
3. Exit
Choose an option: Choose a username: Choose a password (min 4 chars): Full name: Hostel block
(e.g. B3): Room number: Registered successfully. Your student ID is STU-0001 - you can log
in now.

===== Hostel & Mess Complaint System =====
1. Login
2. Register as Student
3. Exit
Choose an option: Username: Password: Welcome, Priya Sharma (STUDENT)

--- Student Menu (Priya Sharma, Room 118) ---
1. File a new complaint
2. View my complaints
3. Leave feedback on a resolved complaint
4. Logout
Choose an option:
Categories:
  1. MESS_FOOD_QUALITY
  2. MESS_HYGIENE
  3. HOSTEL_ROOM_MAINTENANCE
  4. HOSTEL_ELECTRICAL
  5. HOSTEL_PLUMBING
  6. WIFI_NETWORK
  7. SECURITY
  8. OTHER
Choose category number: Priority (1=LOW, 2=MEDIUM, 3=HIGH): Describe the issue: Complaint filed
successfully with ID: CMP-0001

--- Student Menu (Priya Sharma, Room 118) ---
1. File a new complaint
2. View my complaints
3. Leave feedback on a resolved complaint
4. Logout
Choose an option:
===== Hostel & Mess Complaint System =====
1. Login
2. Register as Student
... (truncated for report length) ...

```

9.2 Admin session - status updates and analytics summary

Starting Hostel & Mess Complaint and Feedback System...

```

===== Hostel & Mess Complaint System =====
1. Login

```

2. Register as Student
3. Exit

Choose an option: Username: Password: Welcome, Chief Warden (ADMIN)

--- Admin Dashboard (Chief Warden) ---

1. View all complaints
2. View complaints by status
3. Update complaint status
4. Run escalation check now (threshold: 3 days)
5. View analytics summary
6. Logout

Choose an option: [CMP-0001] MESS_FOOD_QUALITY | HIGH | PENDING | opened
2026-09-12 | | by Priya Sharma

--- Admin Dashboard (Chief Warden) ---

1. View all complaints
2. View complaints by status
3. Update complaint status
4. Run escalation check now (threshold: 3 days)
5. View analytics summary
6. Logout

Choose an option: Complaint ID: New status (PENDING / IN_PROGRESS / RESOLVED): Remarks
(optional): Updated.

--- Admin Dashboard (Chief Warden) ---

1. View all complaints
2. View complaints by status
3. Update complaint status
4. Run escalation check now (threshold: 3 days)
5. View analytics summary
6. Logout

Choose an option: Complaint ID: New status (PENDING / IN_PROGRESS / RESOLVED): Remarks
(optional): Updated.

--- Admin Dashboard (Chief Warden) ---

1. View all complaints
2. View complaints by status
3. Update complaint status
4. Run escalation check now (threshold: 3 days)
5. View analytics summary
6. Logout

... (truncated for report length) ...

10. Testing Approach

Given the console-based, single-user nature of the application, testing was performed as scripted manual test passes - piping a sequence of menu choices into the compiled program and inspecting both the console output and the resulting data files. This validated the full path from user input through the service layer to persistence and back.

Test cases exercised

#	Scenario	Expected result	Result
1	Register a new student, then log in with those credentials	Successful, student menu shows the correct name and room	Pass
2	File a complaint with a valid category/priority/description	Complaint is created as PENDING with a generated ID (e.g. CMP-0001)	Pass
3	File a complaint with an empty description	Rejected with <code>InvalidInputException</code> , no record written	Pass
4	Admin updates a complaint to IN_PROGRESS, then RESOLVED	Complaint is updated with new status and timestamp is set	Pass
5	Student submits feedback (rating + comment)	Feedback is recorded in <code>FeedbackData.txt</code> , linked by complaintId	Pass
6	Student attempts feedback on a non-resolved complaint	Rejected with a clear validation message	Pass
7	Complaint's <code>createdAt</code> manually backdated	Complaint is flagged as <code>ESCALATED</code> in dashboard	Pass
8	Admin views analytics summary after one resolved complaint	Average solution time and category counts reflect the complaint	Pass
9	Malformed row manually inserted into a data file	Row is skipped with a console warning instead of crashing the app	Pass

All nine scenarios passed against the compiled build (OpenJDK 21). Full reproduction steps for each are listed in the "Testing" section of the repository README, so the evaluator can re-run them independently.

11. Challenges Faced

- **Keeping persistence dependency-free** - the submission rules require the project to run from the command line with no GUI-based setup. This ruled out JDBC + a database server as the default storage, so a small but robust flat-file storage utility (`FileStorageUtil`) was built instead, with clear seams (one class) so it could later be swapped for JDBC.
- **Serializing subclasses generically** - Student and Admin both need to write themselves to the same `users.txt` file with different extra fields. This was solved by making `toCsvLine()` abstract on `User` and having each subclass prefix its row with a role tag (STUDENT/ADMIN) that `AuthService` uses to pick the right constructor when reloading.
- **Testing time-based logic without waiting days** - the escalation rule only triggers after real time has passed. This was addressed by writing the check against `Duration.between(createdAt, now)` so it could be tested deterministically by backdating a complaint's stored `createdAt` timestamp instead of waiting.
- **Keeping the UI layer thin** - it was tempting to put validation directly in the console menus. Pushing all validation into the service layer (via `InvalidInputException`) instead means the same business rules would apply unchanged if a GUI or web UI were added later.

12. Learnings & Key Takeaways

- Layering an application (UI / service / model / util) even in a small console project makes it much easier to reason about where a bug or a new feature belongs.

- Abstract classes are a natural fit whenever two or more entities share both data and behaviour but diverge in role-specific fields - here, User modelling Student and Admin.
- Designing around a single configurable constant (`ESCALATION_THRESHOLD_DAYS`) for a business rule makes the system easier to tune and to explain to a non-technical stakeholder (e.g. a warden).
- A deliberately simple persistence format (pipe-delimited text) is enough to demonstrate real CRUD and reporting behaviour, and keeps the project's only external requirement down to a JDK - directly satisfying the command-line executability requirement.

13. Future Enhancements

- **Password hashing** - store salted password hashes (e.g. BCrypt) instead of plain text; flagged in the README as a known limitation of the current scope.
- **Migrate persistence to JDBC + a relational database** using the ER/schema design in Section 6.5, while keeping the service-layer interfaces unchanged.
- **Multi-admin roles** - route complaints to different staff by category (e.g. mess supervisor vs. electrician) rather than a single admin queue.
- **Real notifications** - send an email/SMS on escalation instead of only writing to a local log file.
- **Optional GUI** - a JavaFX front end could sit on top of the existing service layer without any changes to AuthService, ComplaintService, FeedbackService, or ReportService.

14. References

- Oracle, *Java SE 21 Documentation* - docs.oracle.com/en/java/javase/21/
- Oracle, *java.time package (Duration, LocalDateTime)* - docs.oracle.com/en/java/javase/21/docs/api/java.base/java/time/package-summary.html
- Oracle, *java.nio.file package* - docs.oracle.com/en/java/javase/21/docs/api/java.base/java/nio/file/package-summary.html
- VITyarthi, *Build Your Own Project - General Project Instructions & Submission Guidelines* (course-provided PDF).